

CS 4032 – Web Programming



Vue.js

- **Vue.js** (commonly referred to as **Vue**; pronounced [/vjuː/](#), like "view"^[4]) is an [open-source Model–view–viewmodel JavaScript framework](#) for building [user interfaces](#) and [single-page applications](#).^[11]

Vue.js

- Vue (or Vue.js) is an open-source front-end JavaScript framework
- Vue is the **view** layer of an MVC application (Model View Controller)
- Vue is currently [one of the most popular](#) JavaScript libraries/frameworks
- Unlike other popular JavaScript projects, Vue is not backed by a large corporation like React (Facebook) or Angular (Google).
- Vue was originally written by [Evan You](#) and the open-source community.
- Latest Version is released **Version 2.6 2019 Feb**

Vue Features

- **component-based approach to building user interfaces.**
- This allows you to break your application into smaller, reusable pieces (components) which you can then use to build out a more complicated structure.

Working on vue

- Use the vue CLI locally
 - The first is to install the Vue CLI on your computer.
- Use CodeSandbox
 - A simpler way, without having to install anything, is to go to <https://codesandbox.io/s/vue>.
 - CodeSandbox is a cool code editor that allows you build apps in the cloud, which allows you to use any npm package and also easily integrate with Zeit Now for an easy deployment and GitHub to manage versioning.

Vue CLI v3 Installation and Requirements

- Vue CLI v3 requires Node.js 8.9+, but v8.11.0+ is recommended.
- If the vue-cli package is installed globally on your system, you can remove it by running the following command:

```
npm uninstall vue-cli -g
```

- You can now install Vue CLI v3 by simply running the following command from your terminal:

```
npm install -g @vue/cli
```

- You can check the version you have installed by running:

```
vue --version
```

Creating a Vue Project

Using Vue CLI, you can create or generate a new Vue app by running the following command in your terminal:

```
vue create example-vue-project
```

- Navigate inside your project's folder
- Next, run the following command to serve your project locally:

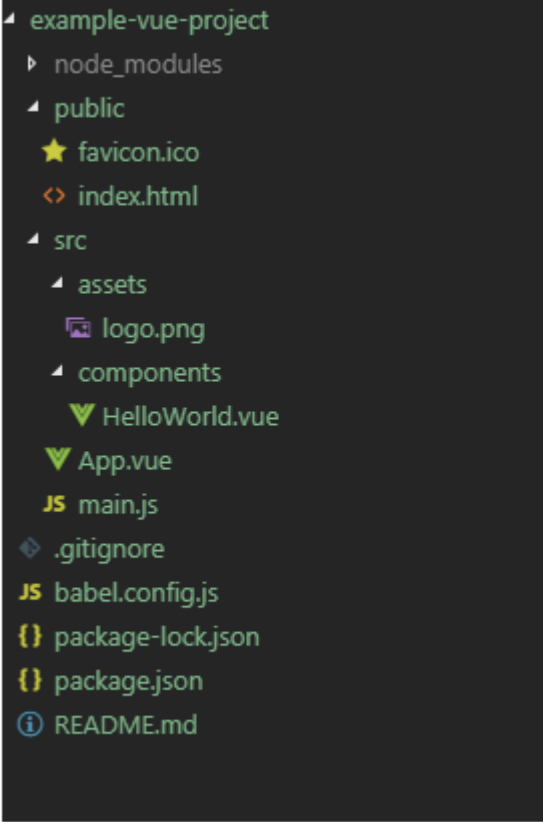
```
npm run serve
```

- The command will allow you to run a local development server from the **http://localhost:8080** address.
- When you've finished developing your project, you can use the following command to build a production bundle:

```
npm run build
```

Vue Application Folder Structure

- index.html
 - main app file, includes one element which will be attached to DOM.
- src/App.vue
 - App.vue is a Single File Component. It contains 3 chunks of code: HTML, CSS and JavaScript tags as
 - <template>
 - <script>
 - <style>
 - We also Import a component from the components/HelloWorld.vue file
- src/main.js
 - This is entry point where we add the vue and render the App to our app div in index.html
- src/components/HelloWorld.vue
 - This folder contains the Vue components.

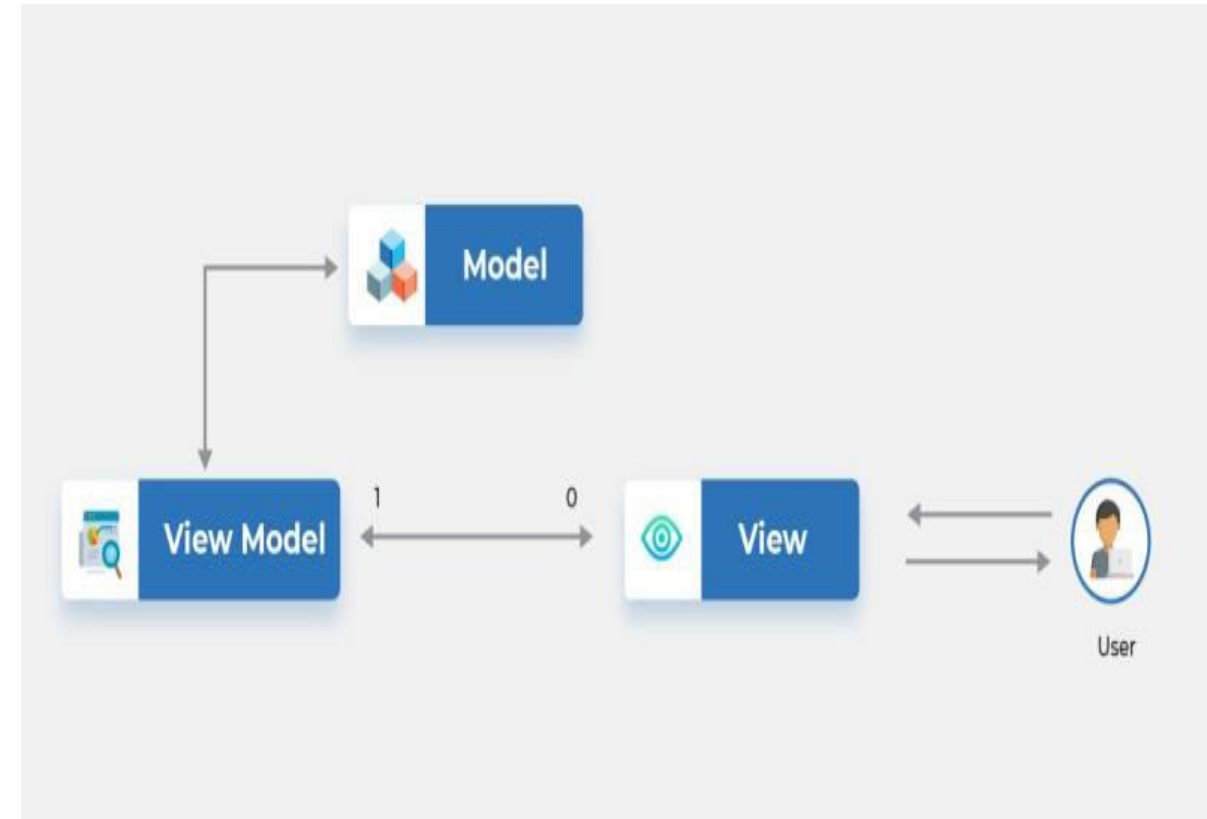
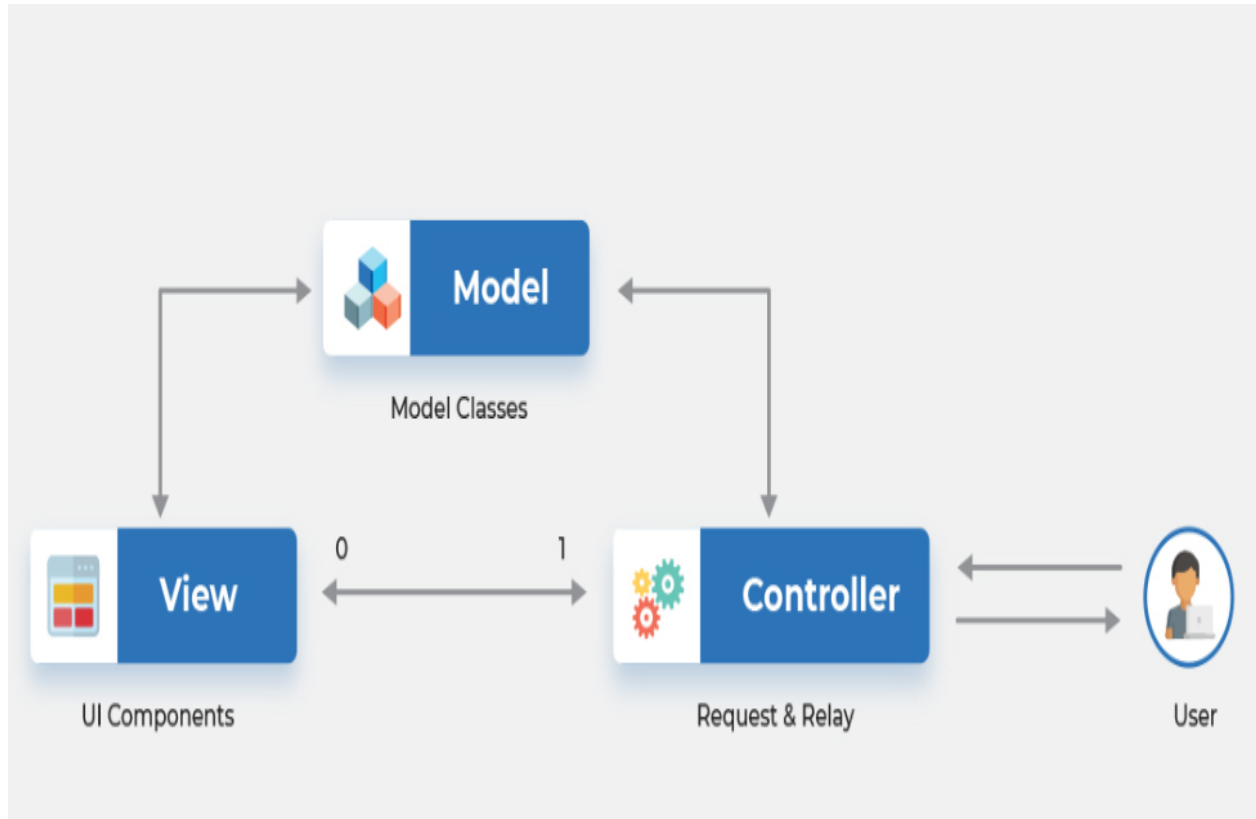


```
example-vue-project
├── node_modules
├── public
│   ├── favicon.ico
│   └── index.html
├── src
│   ├── assets
│   │   └── logo.png
│   ├── components
│   │   └── HelloWorld.vue
│   └── App.vue
├── main.js
├── .gitignore
├── babel.config.js
├── package-lock.json
├── package.json
└── README.md
```


Model-view-viewmodel

- MVVM has been defined from MVC.
- MVVM pattern supports two-way data binding between View and View-Model.
- It allows automatic propagation of modifications inside View-Model to the View.
- Usually, the view-model makes use of observer pattern to make changes in the View-model to the Mode.

MVC Vs. MVVM



Anatomy of a Vue file

- Anything else we make will be a .vue file, which always consists of three things:
 - <template>
 - <script>
 - <style>
- And looks like this:

```
example.vue

<template></template>

<script>
  export default {
    name: 'component-name',
  }
</script>

<style scoped></style>
```

Vue Directly in HTML

- The quickest way start learning React is to write React directly in your HTML files.

```
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <meta name="viewport" content="width=device-width,initial-scale=1.0" />
  <script src="https://cdn.jsdelivr.net/npm/vue/dist/vue.js"></script>
  <title>Vue App</title>
</head>
<body>
  <div id="app"> {{message}}  // Vue.js uses double braces { { } } as place-holders for data
</div>
<script>
  const App = new Vue({
    el: '#app',
    data: {
      message: 'Hello Vue!',
    },})
</script>
</body>
</html>
```

Vue Object

- Before we can do anything with Vue, we need to instantiate a [new Vue instance](#).
- A new Vue object is created with **new Vue()** or **View-Model**
- The property **el**: binds the new Vue object to the HTML element with **id="app"**.

```
<html>
<script src="https://cdn.jsdelivr.net/npm/vue"></script>
<body>
<div id="app">
  <h1>{{ message }}</h1>
</div>

<script>
var myObject = new Vue({
  el: '#app',
  data: {message: 'Hello Vue!'}
})
</script>
</body>
</html>
```

Vue Directives

- Vue.js lets you **extend HTML** with **HTML attributes** called **directives**
- These **Vue directives** offers **functionality** to HTML applications.
- Vue.js directives are **HTML attributes with the prefix v-**
- Vue.js provides
 - **built-in** directives
 - **User defined** directives

Vue built in Directives

- **v-model** : binds the value of HTML elements to application data
- **v-show** : used to conditionally display an element (css based)
- **v-if** : used to conditionally render an element
- **v-else-if** : serves as an “else if block” for v-if and can be chained multiple times
- **v-else** : displays an element when v-if evaluates to false
- **v-for** : iterate over a range, an array, an object, or an array of objects
- **v-on** : listen to various DOM events (click, keyup, submit, etc..)
- **v-bind** : used to dynamically bind one or more attributes, or a component prop, to an expression
- **v-text** : same as {{ interpolation }} and updates the element's textContent

Vue.js Two Way Data Binding

- The two-way data binding is a mechanism where UI fields are bound to model dynamically.
- If the UI components change, model data is also changed accordingly.

v-model

- **v-model** directive is used to establish two-way data binding
- In Example
 - Two way data binding is established with data object's userName property.
 - userName is also displayed in a span using interpolation.
 - when we update the name in the input field, the change is reflected in the span element.

Say your name:
Hello **Web Programmer** How are you today?

Index.html

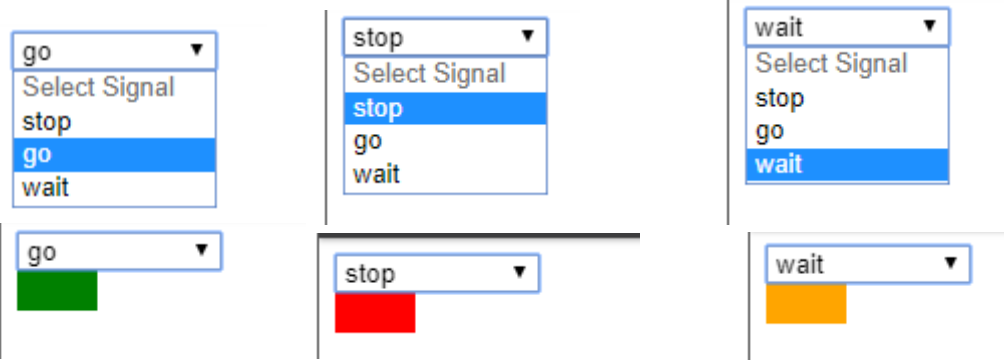
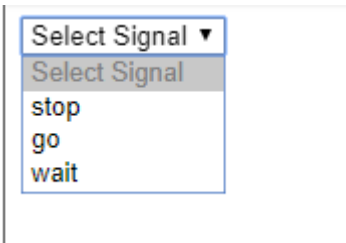
```
<div id="app">
  <span>Say your name: </span>
  <input type="text" v-model="userName" />
  <br/>
  <span>Hello <strong>{{ userName }}</strong></span>
  How are you today?
</div>
```

Main.js

```
new Vue({
  el: '#app',
  data: {
    userName: 'Web Programmer'
  }
})
```

v-bind

- **v-bind** is used to dynamically bind one or more attributes, or a component prop, to an expression
- It can also be used for style binding as shown in the example



```
<div id="app">
  <select v-model="signal">
    <option disabled value="">Select Signal</option>
    <option>stop</option>
    <option>go</option>
    <option>wait</option>
  </select>
  <div v-bind: class="{ 'red': signal === 'stop', 'orange': signal ===
'wait', 'green': signal === 'go'}, 'signal'"></div>
</div>
<style>
  .signal {
    height: 20px;
    width: 40px;
  }
  .red {
    background: red;
  }
  .orange {
    background: orange;
  }
  .green {
    background: green;
  }
</style>
```

Main.js

```
new Vue({
  el: '#app',
  data: {
    signal: ''
  }
})
```

Conditionals Directives

- Vue offers the standard conditionals as directives which can be used in rendering. There are 4 conditional directives available:
- **v-if**
- **v-else**
- **v-if-else**
- **v-show**

Conditional Directives Example

```
<div id="app">
  <span v-if="weather === 'sunny'">Its sunny
  day</span>
  <span v-else-if="weather === 'cloudy'">Its cloudy
  day</span>
  <span v-else>No idea about weather</span>
</div>

<script>
myObject = new Vue({
  el: '#app',
  data: {
    weather: 'cloudy'
  }
})
</script>
```

```
<div id="app">
  <span v-show="weather === 'sunny'">Good</span>
</div>

<script>
myObject = new Vue({
  el: '#app',
  data: {
    weather: 'sunny'
  }
})
</script>
```

v-for : Vue Lists

- Lists are very common way to display information in applications.
- Vue comes with excellent support for displaying lists.
- It provides a **v-for** directive which can be used to loop through collections in templates and render required elements.

v-for

- In the example below we are rendering a list of Australian states.

```
Html
<div id="app">
  <ul>
    <li v-for="state in states">
      {{ state.code }} - {{ state.name }}
    </li>
  </ul>
</div>
```

- NSW - New South Wales
- VIC - Victoria
- ACT - Australian Capital Territory
- QLD - Queensland
- NT - Northern Territory
- WA - Western Australia
- SA - South Australia
- TAS - Tasmania

```
new Vue({
  el: '#app',
  data: {
    states: [
      { code: 'NSW', name: 'New South Wales'},
      { code: 'VIC', name: 'Victoria'},
      { code: 'ACT', name: 'Australian Capital Territory'},
      { code: 'QLD', name: 'Queensland'},
      { code: 'NT', name: 'Northern Territory'},
      { code: 'WA', name: 'Western Australia'},
      { code: 'SA', name: 'South Australia'},
      { code: 'TAS', name: 'Tasmania'}
    ]
  }
})
```

Vue Events

- event can be any of the standard JavaScript events like
- **onfocus, onblur, onchange, onselect, onmouseover, onclick, onload, onsubmit etc.**
- The equivalent in Vue would be something like
- **v-on:focus="function",**
- **v-on:blur="function",**
- **v-on:change="function",**
- **v-on:select="function",**
- **v-on:mouseover="function",**
- **v-on:click="function",**
- **v-on:load="function",**
- **v-on:submit="function".**

Vue Events

- Once you decide which event you want to respond to, you think of a custom function name that gets passed to it.
- This is what comes as the attribute name in **v-on:event="function"**
- This custom named function then gets declared in the methods object of the Vue instance. So just to tie this together, instead of one of these first two:

native js

```
1 <div id="element" onclick="alert('you just clicked this');"></div>
```

jquery

```
1 $("#element").click(function () {  
2   alert("you just clicked this");  
3 });
```

html

```
1 <div id="element" v-on:click="myfunction"></div>
```

Vue js

```
1   methods: {  
2     myfunction: function () {  
3       alert('you just clicked this');  
4     }  
5   }
```


v-on - Vue Event Handling

- These events can be handled and we can take actions.
- The way to handle events in Vue is by using **v-on** directive.

```
<div id="app">
  <input type="text" v-model="num" />
  <button v-on:click="getFactorial">Get Factorial</button>
  <span>Factorial of {{ num }} is {{ factorial }}</span>
</div>
```

Factorial of 6 is 720

```
new Vue({
  el: '#app',
  data: {
    num: 0,
    factorial: 1
  },
  methods: {
    getFactorial: function (event) {
      let num = this.num
      this.factorial = computeFactorial(this.num)
    }
  }
})

const computeFactorial = (num) => {
  if (num === 0) {
    return 1;
  }
  return num * computeFactorial(num - 1);
}
```

User Defined/Custom Directives

- Vue also allows us to register your own custom directives.
- We can register a global custom directive with the `Vue.directive()` method, passing in a directiveID followed by a **definition object**.
- Once registered, Custom directives can be used in Vue.js templates.
- A **definition object** can provide **several optional hook functions**.

```
Vue.directive('my-directive', {  
  bind: function (value) {  
    // do preparation work add event listeners  
  },  
  update: function (value) {  
    // do something based on the updated value  
  },  
  unbind: function () {  
    // do clean up work-remove event listeners added in bind()  
  }  
})
```

```
<div v-my-directive="someValue"></div>
```

A directive has five possible hooks:

- `bind(element, binding, vnode)` - Called when the directive is first bound to the component or element.
- `inserted(element, binding, vnode)` - Called when the component or element is inserted into its parent node. It may not be in the DOM yet.
- `update(element, binding, vnode, oldVnode)` - Called when the containing component has updated but potentially before its children do.
- `componentUpdated(element, binding, vnode, oldVnode)` - Called when the containing component has updated but after its children do.

Vue Filters

- A Vue.js filter is essentially a function that takes a value, processes it, and then returns the processed value onto the web page.
- In the markup it is denoted by a single pipe (|) and can be followed by one or more arguments.
- Let us apply formatting and transformations to a value that's used in a template interpolation
- Filters do NOT change the data itself but rather change the output to the browser by returning a *filtered* version of that data.
- Filters can be categorized in two forms:
 - Built in Filters
 - Custom Filters

Install Filters

1. Run this on command prompt to install Vue Filters Package

```
npm install vue2-filters
```

2. Direct Include

- Simply include vue2-filters after Vue and it will install itself automatically:

```
<script src="vue.js"></script>  
<script src="vue2-filters.min.js"></script>
```

3. Using CDN

```
<script src="https://unpkg.com/vue/dist/vue.min.js"></script>  
<script src="https://cdn.jsdelivr.net/npm/vue2-filters/dist/vue2-filters.min.js"></script>
```

Built in Filters

- Various built in filters are available in the Vue framework itself that cover many common use cases such as
 - **capitalize,**
 - **uppercase, lowercase,**
 - **currency, pluralize,**
 - **json,**
 - **key,**
 - **filterBy, and orderBy.**

- Syntax:

```
{{ data | builtin filter }}
```

Built in Filters - Example

```
<div id="app" >
  <span>Hello <strong>{{ userName | uppercase }}</strong>
</span>
</div>
```

```
<script>
new Vue({
  el: '#app',
  data: {
    userName: 'Web Programmer'
  }
});
```

Hello **WEB PROGRAMMER**

```
{{ msg | truncate(10) }} // 'lorem ipsum dolor' => 'lorem ipsu...'
```

```
{{ 123456 | number('0,0') }} // => 123,456
```

```
{{ 0.974878234 | percent(3) }} // => 97.488%
```

```
{{ amount | currency }} // 12345 => $12,345.00
```

```
{{ amount | currency('£') }} // 12345 => £12,345.00
```

```
{{ date | ordinal }}
```

```
// 1 => 'st'
// 2 => 'nd'
// 3 => 'rd'
// 4 => 'th'
// 5 => 'th'
```

```
{{ date | ordinal({ includeNumber: true }) }}
```

```
// 1 => '1st'
// 2 => '2nd'
```

Custom Filters

- There are two types of custom filters
 1. **Global Filters** We can access global filters throughout our Vue app and they should be declared before the Vue instance.
 2. **Local Filters** are declared inside the Vue instance. We can only access local filters within a component.
- In both cases, we would pass the value in as a parameter.

```
//global  
Vue.filter('filterName', function(value) {  
  return // thing to transform  
});  
  
//locally, like methods or computed  
filters: {  
  filterName(value) {  
    return // thing to transform  
  }  
}
```


Global Filters

- There are certain situations where a particular filter can be used on several components within your application,
- Especially in those larger projects, where several interactions of the system can occur throughout the project.
- We need to register the Filter at a global level. So in the **main.js** file and register the filter.

```
Vue.filter(' mylowercase', function (value)
{
  return value.toLowerCase();
});
```

```
// USAGE
<div id="app">
  <span>
    {{ data | mylowercase }}
  </span>
</div>
```

Local Filter -Example

```
new Vue({
  el: '#app',
  data() {
    return {
      customer1total: 35.43
    }
  },
  filters: {
    tip15(value) {
      return (value*.15).toFixed(2)
    },
    tip20(value) {
      return (value*.2).toFixed(2)
    },
    tip25(value) {
      return (value*.25).toFixed(2)
    }
  }
});
```

```
<div id="app">
  <h2>Tip Calculator</h2>
  <p><strong>Total: {{ customer1total }}</strong></p>
  <p>15%: {{ customer1total | tip15 }}</p>
  <p>20%: {{ customer1total | tip20 }}</p>
  <p>25%: {{ customer1total | tip25 }}</p>
</div>
```

Tip Calculator

Total: 35.43

15%: 5.31

20%: 7.09

25%: 8.86

Computed Properties

- Computed properties are like normal methods in Vue, **but with the caching ability on the reactive dependencies.**
- Computed properties allow us to have model-specific, complex values computed for the view.
- These values will be bound to the dependency values and only update when required.
- Computed Properties are passed into the Vue instance as an option. They do the same thing as methods i.e. they invoke a function.
- So why do we have them?
 - Because they provide caching.

Computed Properties -Example

```
new Vue({  
  el: '#app',  
  data: {  
    celsius: 0  
  },  
  computed: {  
    celsiusToFahrenheit: function() {  
      return this.celsius * 9 / 5 + 32;  
    }  
  }  
})
```

```
<div id="app">  
  <input type="text" v-model="celsius" />  
  <p>  
    {{ celsius }} Celsius = {{ celsiusToFahrenheit }} Fahrenheit  
  </p>  
</div>
```

39 Celsius = 102.2 Fahrenheit

Search filter based on input field -Example

```
const app = new Vue ({
  el: '#app',
  data: {
    search: "",
    userList: [
      {id: 1, name: "John Doe" },
      { id: 1, name: "Mike Hang"},
      { id: 1, name: "Patrick Dezi"}
    ] },
  computed: {
    filteredAndSorted(){
      // function to compare names
      function compare(a, b) {
        if (a.name < b.name) return -1;
        if (a.name > b.name) return 1;
        return 0;
      }
      return this.userList.filter(user => {
        return user.name.toLowerCase().includes( this.search.toLowerCase())
      }).sort(compare)} }
  })
```

Search filter based on input field

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/vue/2.4.2/vue.js"></script>
<div id="app">
  <div class="search-wrapper">
    <input type="text" v-model="search" placeholder="Search title.."/>
    <label>Search Users:</label>
  </div>
  <ul>
    <li v-for="user in filteredAndSorted">{{user.name}}</li>
  </ul>
</div>
```

Search Users:

- John Doe
- Mike Hang
- Patrick Dezi

Search Users:

- John Doe

Search Users:

- Patrick Dezi

Vue Watchers

- A watcher is a special Vue.js feature that allows you to spy on one property of the component state, and run a function when that property value changes.
- Watchers provide an additional level of control by allowing us to listen for changes to a property or change in a data model.
- We can use watchers for more complex requirements, for example:
 - Async operations
 - Setting intermediate values
 - Limiting the number of times an operation gets called (ex: Debounce an input event)

Vue Watcher Example

```
new Vue({
  el: "#app",
  data: {
    age: 19,
    alert: ''
  },
  methods: {
    changeAge(){
      this.age++;
    }
  },
  watch: {
    age(){
      this.alert = 'age changed!'
    }
  }
})
```

```
<div id="app">
  <h2>Click the button</h2>
  <p>{{age}}</p>
  <button @click='changeAge'>Click me</button>
  <p>{{alert}}</p>
</div>
```

Click the button

19

Click me

Click the button

20

Click me

age changed!

Vue Components

- Components are reusable Vue instances with a name.
- We can define a custom html tag, and implement what that tag does in Vue.js.
- This custom tag can be reused as an html tag.

```
Vue.component('app-header', {  
  template: "<li>This is the application's header</li>"  
})
```

```
<div>  
  <app-header></app-header>  
</div>
```

The building blocks of a component

- A component can accept the `el`, `props` and `template` properties.
- `el` is only used in root components initialized using `new Vue({})`, and identifies the DOM element the component will mount on.
- `props` lists all the properties that we can pass down to a child component
- `template` is where we can set up the component template, which will be responsible for defining the output the component generates.

Component

- A component accepts other properties:
- `data` the component local state
- `methods`: the component methods
- `computed`: the computed properties associated with the component
- `watch`: the component watchers

Components Example

```
Vue.component('author-detail', {
  template:
    <div>
      <h2>{{ owner.name }}</h2>
      <p>{{ owner.email }}</p>
    </div>
  ,
  props: {
    owner: {
      type: Object,
      required: true
    }
  })
new Vue({
  el: '#app',
  data() {
    return {
      author: {
        name: 'John Doe',
        email: 'jdoe@example.com'
      }
    }
  }
})
```

```
<div id="app">
  <author-detail :owner="author">
  </author-detail>
</div>
```

John Doe

jdoe@example.com

Vue Props

- Props is the short form for properties. Properties are attributes of a component.
- In fact, props are how components talk to each other. A tag in HTML such as `<img>` has an attribute, a.k.a prop called `src` that points to the location of an image.
- Pass data from the parent scope into child components. W

Vue Components and props example

```
Vue.component('child', {
  props: {
    count: {
      type: Number,
      required: true
    }
  },
  template: `<div class="num">{{ count }}</div>`
})

new Vue({
  el: '#app',
  data() {
    return {
      count: 0
    },
  },
  methods: {
    increment() {
      this.count++;
    },
    decrement() {
      this.count--;
    }
  })
})
```

```
<div id="app">
  <h3>
    <button @click="increment">+</button>
    Adjust the data
    <button @click="decrement">-</button>
  </h3>
  <h2>This is the app data: <span class="num">{{
count }}</span></h2>
  <hr>
  <h4><child count="3"></child></h4>
  <p>This is a child counter that is using a static
integer as props</p>
  <hr>
  <h4><child :count="count"></child></h4>
  <p>This is the same child counter and it is using
the vue instance data as props</p>
</div>
```

Components – Example Output

+

Adjust the data

-

This is the app data: 8

3

This is a child counter that is using a static integer as props

8

This is the same child counter and it is using the vue instance data as props

Local vs Global Component

- Any component created using `Vue.component()` is globally registered.
- We don't need to assign it to a variable or pass it around to reuse it in templates.
- We can encapsulate components locally by assigning an object that defines the component object to a variable.
- and then make it available inside another component by using the `components` property:

```
const Sidebar = {  
  template: '<aside>Sidebar</aside>'  
}
```

```
new Vue({  
  el: '#app',  
  components: {  
    Sidebar  
  }  
})
```


Component Lifecycle

- Vuejs 2 provides some methods that are triggered at various points from creating a component up until the component is destroyed.
- This is called the Instance Lifecycle, also known as the Component's Lifecycle.
- Each Vue instance goes through a series of initialization steps when it is created

Component Lifecycle

- when Vue creates the component,
- mounts the component to the DOM,
- updates the component and
- destroy the components.
- In order words, Each component has what is known as lifecycle events — its birth, life events like changes and death.
- We can tap into key moments in that lifecycle by implementing one or more lifecycle hooks that gets called by Vue itself thereby giving us the opportunity to add our own code at specific stages of a component lifetime.

Component Lifecycle hooks

- These lifecycle hooks are:
 1. `beforeCreate()`,
 2. `created()`,
 3. `beforeMount()`,
 4. `mounted()`,
 5. `beforeUpdate()`,
 6. `updated()`,
 7. `activated()`,
 8. `deactivated()`,
 9. `beforeDestroy()`
 10. `destroyed()`.

Component Lifecycle hooks

```
<script>
export default {
  data() {
    return {
      counter: 0
    }
  },

  beforeCreate() {
    console.log('I am the first hook to get called!');
    console.log(`You can see that my data property is ${typeof this.counter}`);
  }
}
</script>
```

Vue Routing

- Routing is the part that makes a URL change when you click something on a page and helps to show the correct view or HTML when you navigate the specific URL.

Vue Router Installation

- We can use Vue Router via a script tag, you can include using

```
<script src="https://unpkg.com/vue-router"></script>
```

- We can install the vue-router by default while we were creating a new project,
- OR we can install it later on by using this command in the working directory
app

```
npm install vue-router --save
```

Vue Router Modes

- Vue Router has two modes for its URL.
 1. **Hash Mode (default)** - uses URL hash to simulate a URL; e.g. `mypage.com/#profile`
 2. **History** - looks like a typical URL and uses `history.pushState` to avoid page reloads; e.g. `mypage.com/profile`

Router Components

- **Rendering a Route**

- The `<router-view>` component **is the place** where the Vue Router **will put the content that matches the current URL**

Linking To A Page

- Vue Router uses special link elements called `<router-link>` that have an attribute that map to a component.
 - A `<router-link>` component renders a tag by default .
-
- Every time the route changes, either by clicking a link or by changing the URL, a `router-link-active` class is added to the element that refers to the active route, allowing you to style it.

Vue Router –Basic Routing

- Now make two changes inside the **src >> main.js** file
 1. **import VueRouter from 'vue-router'**
 2. Add **Vue.use(VueRouter) ;**

After the above changes, application have access to these objects:

- **this.\$router** is the router object
- **this.\$route** is the current route object
- **this.\$router** and **this.\$route** will be used for Dynamic Routing.

Basic/Static Routing Example

- The next step is to create three new components inside the **src >> components** folder.

```
// Home.vue

<template>
  <div>
    Home
  </div>
</template>
<script>
export default {

}
</script>
```

```
// Register.vue

<template>
  <div>
    Register
  </div>
</template>
<script>
export default {

}
</script>
```

```
// Login.vue

<template>
  <div>
    Login
  </div>
</template>
<script>
export default {

}
</script>
```

Basic/Static Routing Example

- Inside the **src** folder, create one file called **routes.js** and add the following code.

```
// routes.js

import Home from './components/Home.vue';
import Register from './components/Register.vue';
import Login from './components/Login.vue';

const routes = [
  { path: '/', component: Home },
  { path: '/register', component: Register },
  { path: '/login', component: Login },
];

export default routes;
```

Basic/Static Routing Example

- The next step is to create a VueRouter instance and pass the routes object.
- So, we need to import **routes.js** file inside the **main.js** file

```
// main.js

import Vue from 'vue';
import App from './App.vue';
import VueRouter from 'vue-router';

import routes from './routes';

Vue.config.productionTip = false;

Vue.use(VueRouter);

const router = new VueRouter({routes});

new Vue({
  router,
  render: h => h(App)
}).$mount('#app');
```

Basic/Static Routing Example

- Also, we need to display the routing components based on the routes.
- So we can do that by adding the **<router-view>** component. So let us add that inside **src >> App.vue** file.

```
// App.vue

<template>
  <div id="app">
    <nav>
      <router-link to="/">Home</router-link>
      <router-link to="/register">Register</router-link>
      <router-link to="/login">Login</router-link>
    </nav>
    <router-view />
  </div>
</template>

<script>
export default {
}
</script>
```

Named Route

- Named routes functionality allows to call the name of the route instead of the URL in router-link tags.
- We can add a name to the route by adding a name property to route object.

```
= [  
  { path: '/',      name: 'home',  component: Home},  
  { path: '/about', name: 'about',  component: About},  
  { path: '/contact', name: 'contact', component: Contact}]; const routes
```

- Now in your router-link tags you can change the routes to use the name by using the following code.

```
<router-link :to="{ name: 'about' }">About</router-link>
```

- **benefit** : if your routes change then you don't need to go through all the tags and change the route you just change it in your route object.

Vue HTML 5 History Mode Routing

- We need to modify one line of code inside the **main.js** file, and we are done.

```
// main.js  
  
const router = new VueRouter({mode: 'history', routes});
```

Dynamic Routing

- In vue router, we are rendering the components based on the path matching
- But most apps don't have simple links for the URLs and they're dynamic such as <http://example.com/blog/{postname}>.
- This allows the website to only use one page for the unlimited number of urls.
- Vue Router do it by **defining routes with dynamic segments**.
- Dynamic segment is added by using colon **:nameofthesegment**

```
routes: [  
  { path: '/blog/:postname', component: Blog }  
],
```

```
routes: [  
  { path: '/', component: Home },  
  // dynamic segment `:id` is added to the path  
  { path: '/user/:id', component: User },  
]
```


Dynamic Routing – Access Dynamic Segment

- There are two ways to access dynamic segment inside any component :

1. Access the dynamic segment with **this.\$route.params** in component **blog**

```
const Blog = {  
  template: '<div>Blog post page - {{ $route.params.postname }}</div>'  
}
```

2. a- Pass dynamic segment **as a props** to our component in Routes array

```
routes: [  
  { path: '/blog/:postname', component: Blog , props : true}  
],
```

- b- In blog component, define a prop and access that prop using **{{}}**

```
<template><div> {{ postname }}</div></template>  
<script>  
export default {  
  props: {  
    postname: String  
  }  
</script>
```

Vue Route Object

- **\$route** object refers to a route defined in current URL so **this.\$route** is the current route object.
- Using **\$route object** also exposes other useful information such as
 - **path** - The full path of the route
 - **params** - Key/value of the dynamic segments
 - **query** - Key/value of query string parameters
 - **hash** - The route of the current hash value
 - **matched** - An array of matched routes
 - **name** - The name of the route

Dynamic Routing -Example

```
// Student.vue

<template>
  <div>
    Student ID is: {{ $route.params.id }}
  </div>
</template>
<script>
export default {

}
</script>
```

```
// main.js

import Home from './components/Home.vue';
import Register from './components/Register.vue';
import Login from './components/Login.vue';
import Student from './components/Student.vue';

const routes = [
  { path: '/', component: Home },
  { path: '/register', component: Register },
  { path: '/login', component: Login },
  { path: '/student/:id', component: Student },
];

export default routes;
```

Programmatically Change the Route

So far, we navigated between our different routes using `<router-link>` elements.

- We can redirect the route programmatically using the **Router Object**.
 - For example, when the user successfully logged in, we need to redirect to the home route programmatically.
- Router Programmatically change the Route using Router history.

Router History

- The router history allows you to go back to the last page.
- The history of pages you visit in your web browser is like a stack data structure.
- You can either
 - Push a new record onto the top of the history stack
 - OR you can replace the top record.

Vue Router Object

- We can access the router object using `this.$router` from any component
- when a Vue Router is installed in the root Vue component and it offers many nice features.
- We can make the app navigate to a new route using
- **`this.$router.push()`**
`this.$router.replace()`
`this.$router.go()`
- If you have performed the CRUD operations previously using Vue.js as a frontend then we have used the **`push()`** method to change the route programmatically.

.\$router Object

- **.\$router.push()** is used to go to a new route, adding a new item to the browser history.

```
this.$router.push({ path: 'about' })
```

.\$router.replace() is used to overwrite the top of the the stack.

```
this.$router.replace({ name: 'post', params: { post_slug: 'hello-world' } })
```

.\$router.go() goes back and forth, accepting a number that can be positive or negative to go back in the history

```
this.$router.go(-1) //go back 1 step  
this.$router.go(1) //go forward 1 step
```

Connect Vue With Backend

Various backend could be used for your next Vue.js project

- [Express](#) using Node
 - [Laravel](#)
 - [WordPress](#)
 - [Firebase](#)
 - [Serverless](#)
 - [Django](#)
 - [Rails](#)
- Connecting Express with node is most easy as we have already discussed.

Connect backend

- For Vue.js following options are available:
- [Fetch](#) - [*Browser's built-in fetch API*](#)
- [Axios](#)
- [Jquery Ajax](#)

Fetch vs Axios

- Both are Promise-based
- Fetch is a standard [Web API](#)
 - May not be available on older browsers
 - Available for React Native
- [Axios](#) is a popular JavaScript HTTP client library
 - Treat error status codes as error so it can be handled in `catch()` instead of `then()`

Example

Search Article from Fetch REST API

Fetch API Example - html

```
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>JS Bin</title>
  <script src="https://cdn.jsdelivr.net/npm/vue@2.5.16/dist/vue.js"></script>
</head>
<body>
  <div id="divContent">
    <h1>Article Search Results</h1>
    <form v-on:submit.prevent="search">
      <input type="text" v-model="query">
      <button type="submit">Search</button>
    </form>
    <ul>
      <li v-for="article in articles" v-bind:key="article.source + article.id">
        {{ article.title }}
      </li>
    </ul>
  </div>
</body>
```

Fetch API Example- Js

```
const vm = new Vue({
  el: '#divContent',
  data() {
    return {
      query: 'gene',
      articles: 'loading'
    }
  },
  created() {
    this.search();
  },
  methods: {
    search: function () {
      fetch(`https://www.ebi.ac.uk/europepmc/webservices/rest/search?query=${this.query}&format=json`)
        .then(response => response.json())
        .then(json => {
          this.articles = json.resultList.result;
        });
    }
  }
});
```

Fetch API Example - Output

Output

Article Search Results

- Sequence analysis of the N-acetyltransferase 2 gene (NAT2) among Jordanian volunteers.
- In vitro marker gene expression analyses in human peripheral blood mononuclear cells: A tool to assess safety of influenza vaccines in humans.

Example

Get Todo list from Axios API

Axios – Example

```
<script>
import axios from "axios";

export default {
  data() {
    return {
      todosList: []
    };
  },
  mounted() {
    axios.get("https://jsonplaceholder.typicode.com/todos/")
      .then(response => {
        this.todosList = [...response.data].slice(0, 10)
      })
      .catch(err => {
        // Manage the state of the application if the request
        // has failed
      })
  }
};
</script>
```


Axios – Example

```
<template>
  <div>
    <ul
      class="test-list"
      v-for="todo in todosList"
      :key="todo.id"
    >
      <li class="test-list--item">
        {{ todo.id }}
      </li>
    </ul>
  </div>
</template>
```

Axios – Example output

1	Delectus Aut Autem	Incomplete
2	Quis Ut Nam Facilis Et Officia Qui	Incomplete
3	Fugiat Veniam Minus	Incomplete
4	Et Porro Tempora	Complete
5	Laboriosam Mollitia Et Enim Quasi Adipisci Quia Provident Illum	Incomplete

Vue Readings

- [Vuex \(state management framework\)](#)